

Introduction to DevOps

Lecture 01

Muhammad Kamran

An aerial photograph of a shipping yard. On the left, there are large stacks of intermodal containers in various colors including blue, red, orange, and green. To the right, a long row of containers is neatly stacked along a paved area. Yellow numbers (2050, 2056, 2057, 2058, 2059, 2054) are painted on the pavement next to the containers. The text 'THE BASICS' is centered over the image in a white, sans-serif font.

THE BASICS

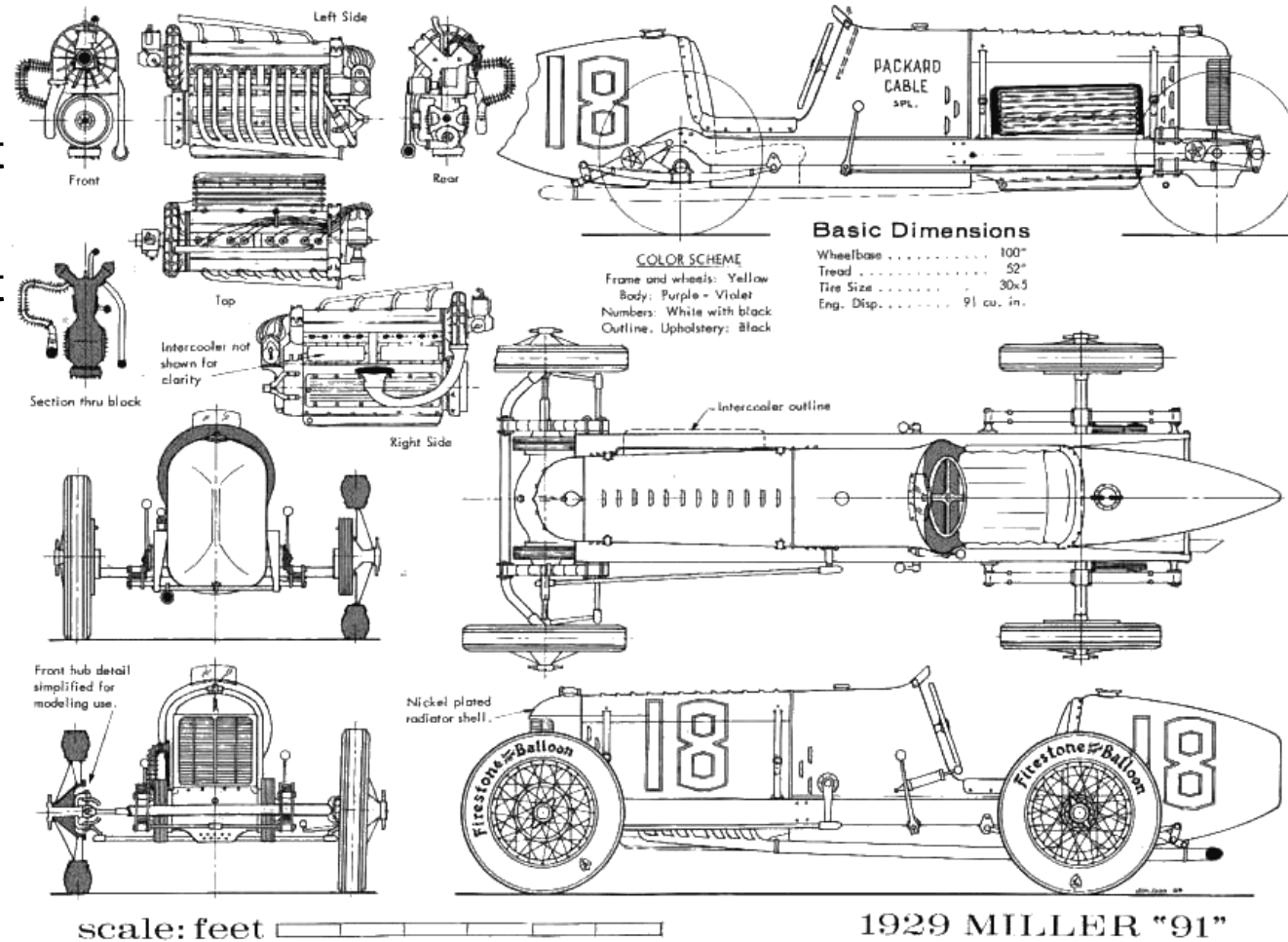
Who is Dev?

Who is Ops?

Traditional Development

The Inventors

- Create new features and functionality in “dev” environment
- Occasionally deliver new product to operators, along with instructions
- May incorporate feedback from operators in future deliveries
- Rewarded for delivering new features



The inventors are responsible for changing the system

Traditional Operations



The Mechanics

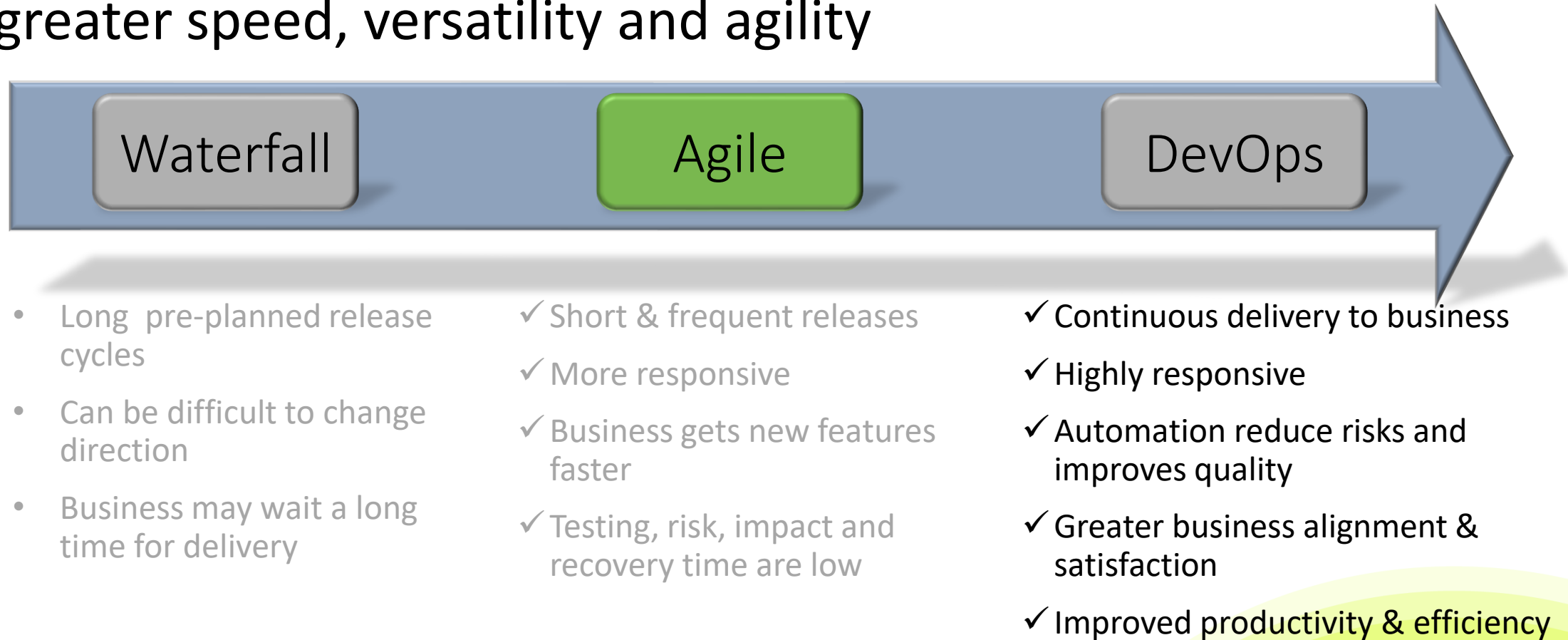
- Receive new products from developers to be installed and operated
- Expected to keep production systems up and running
- Track problems, deployment failures, and system outages
- May provide feedback to the inventors for future consideration
- Penalized for downtime



The mechanics are responsible for keeping the system in operation

Evolution towards agile/DevOps

DevOps is built on agile principles and provides a platform for greater speed, versatility and agility

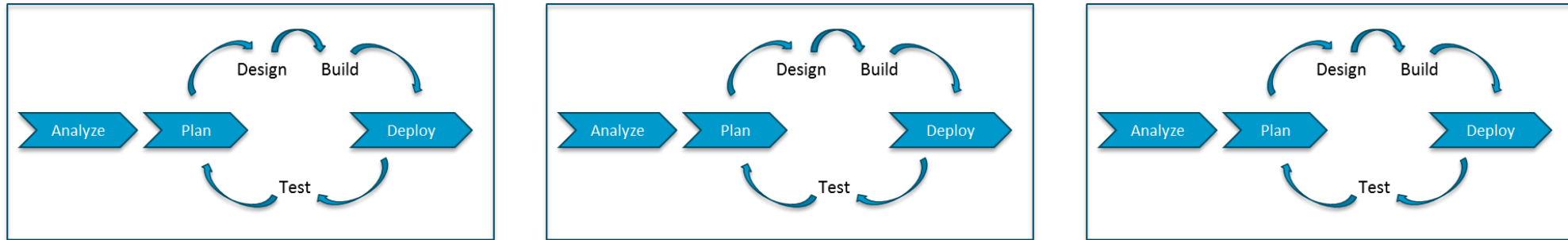


Waterfall vs. Agile Dev Methodologies

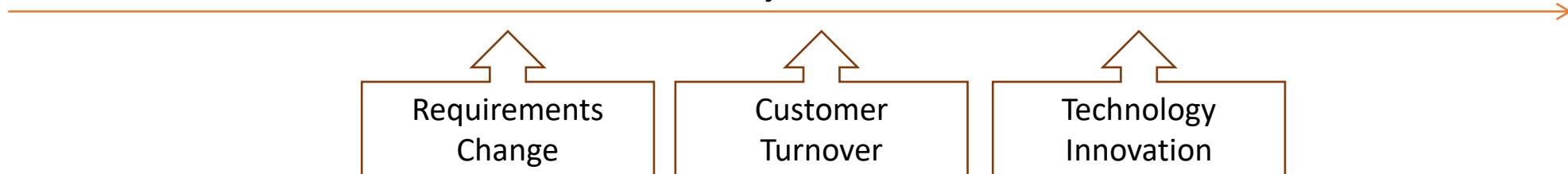
Waterfall



Agile

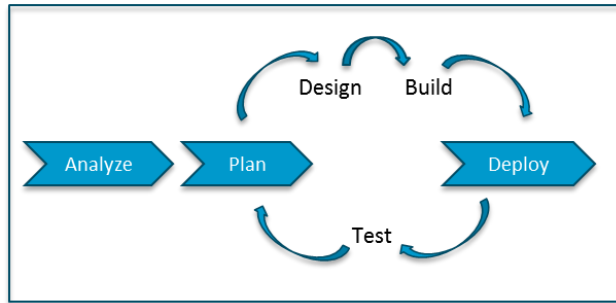


Project Timeline



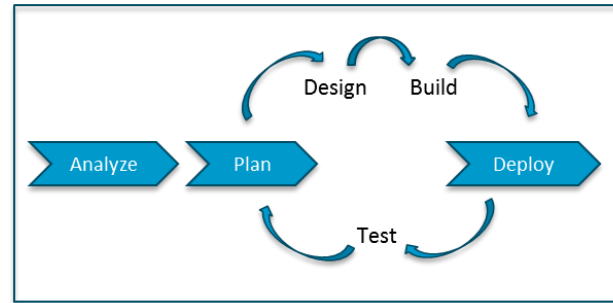
Agile (Dev) + Deploy (Ops) → Automation

DevOps



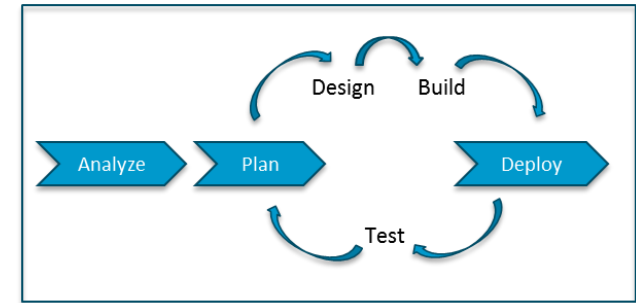
Continuous
Integration

Continuous
Deployment



Continuous
Integration

Continuous
Deployment

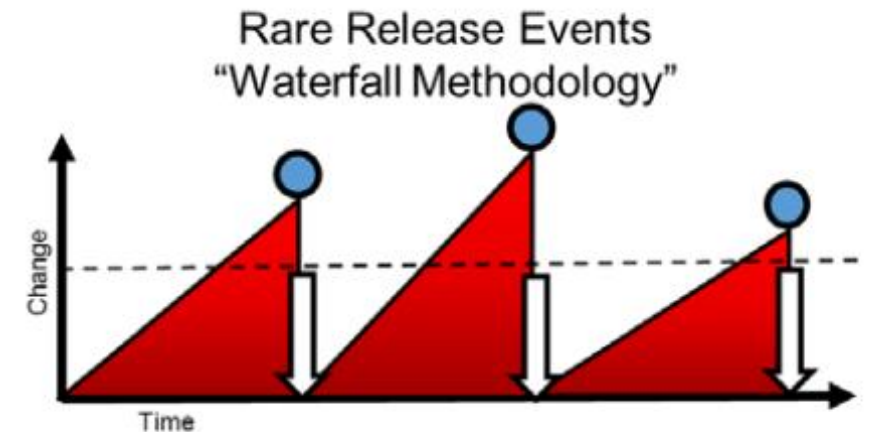


Continuous
Integration

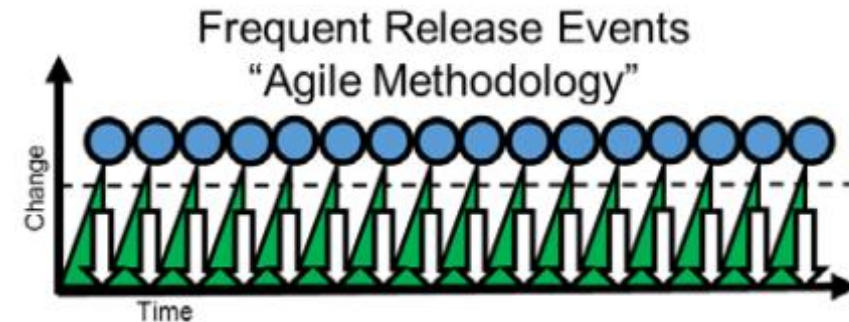
Continuous
Deployment



Big Bang vs. Continuous Integration



- Infrequent Stakeholder Involvement & Feedback
- Monolithic, Big Bang Integration
- Slow Response to Change, Higher Risk



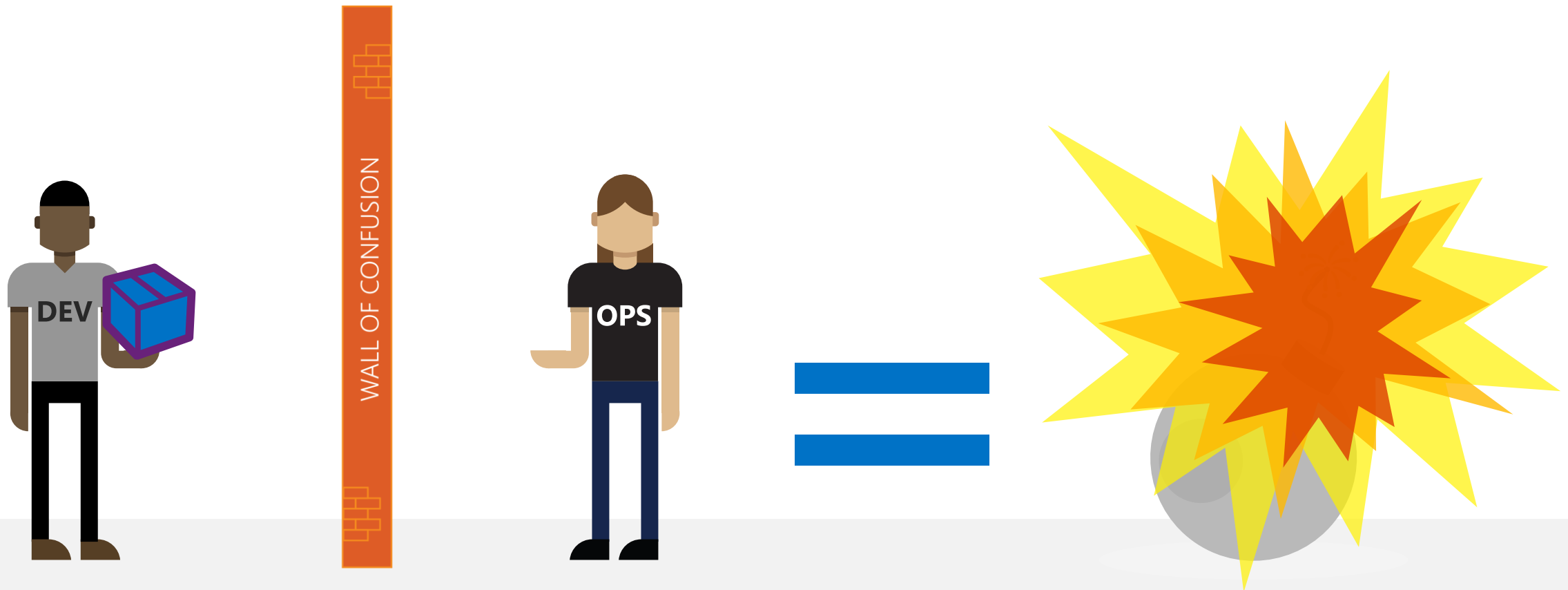
- Frequent Stakeholder Involvement & Feedback
- Continuous Integration
- Rapid Response to Change, Less Risk

What problems led to the creation of DevOps?

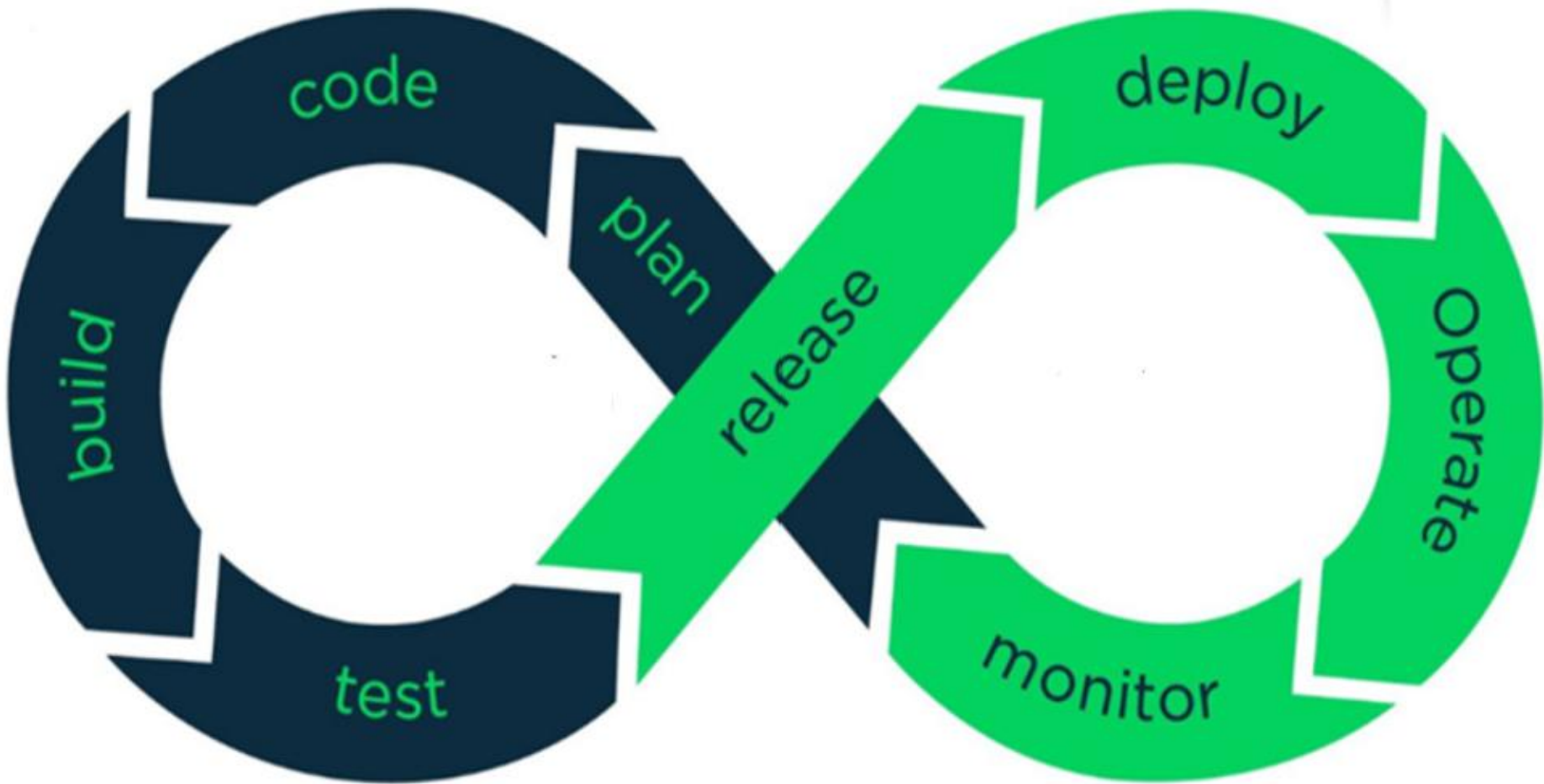
- DevOps implementation has **increased** the *rate of software delivery* and *revenue* for business stakeholders. The following listed reasons are the most considerable ones that led to the creation of DevOps.
 - Before DevOps, operation and development teams were working in an *isolated environment*.
 - *Testing* and *Deployment* activities mostly were performed in an isolated manner after the design-build step, and *they took more time than the actual project completion time*.
 - Team members usually spend a *large amount of time* deploying, testing, designing, and building the projects
 - *Human production errors were deployed* during manual code conduction.
 - Operations and coding teams generally had different timelines and *did not have proper synchronization* which results in further delay.



Traditional Development and Operations



Development



Operations

What is DevOps

- DevOps is a modern approach to software development that brings development and operations teams together to deliver applications faster and more reliably.
- Encourages close collaboration between development and operations teams.
- Automates build, test, and deployment processes to reduce errors.
- Enables faster and more frequent software releases.
- Improves system reliability, monitoring, and feedback loops.

DevOps Stages (Plan)

- Convert business needs into **clear, testable work** (user stories, tasks, acceptance criteria)
- Maintain a **prioritized backlog** and define “Definition of Ready” / “Definition of Done”
- Plan for **operability early**: availability, performance, scalability, logging/metrics needs
- Decide release strategy: incremental delivery, MVP slicing, risk reduction
- **Popular tools**: Jira, Azure Boards, Trello; Confluence/Notion/Wikis for documentation

DevOps Stages (Code)

- Build features in **small, reviewable changes** (frequent commits, pull requests)
- Use **version control as the system of record** (code + configs + pipeline definitions)
- Practices: branching strategy, code review, coding standards
- “Shift-left” mindset: developers consider **deployability + observability** while coding
- **Popular tools:** Git; GitHub/GitLab/Bitbucket; pre-commit hooks, linters (ESLint, Pylint)

DevOps Stages (Build)

- Turn source code into **immutable artifacts** (binaries/packages/container images)
- Make builds **repeatable** (same input → same output) and fast (cache dependencies)
- Embed metadata: version, commit hash, build number (traceability)
- Store artifacts centrally for promotion across environments
- **Popular tools:** Jenkins / GitHub Actions / GitLab CI; Maven/Gradle/npm;

DevOps Stages (Test)

- Automate tests to validate changes **before** they reach production
- Typical layers: unit → integration → API/contract → end-to-end/UI
- Gate the pipeline with **quality checks** (coverage thresholds, linting, static analysis)
- Prefer isolated test environments for reliability
- Popular tools:** JUnit/PyTest; Selenium (UI); Postman(API); SonarQube (quality); test reporting in CI

DevOps Stages (Deploy)

- “Release” = **approve/promote a tested artifact** as a candidate for deployment
- Apply versioning (semantic versioning), generate release notes, tag commits
- Introduce controls without slowing flow: lightweight approvals, automated evidence, audit trails
- Use feature flags to decouple “deploy” from “expose to users” (reduce risk)
- **Popular tools:** Jenkins pipelines (promotion stages), GitHub Releases/Tags, artifact repository promotion

DevOps Stages (Operate)

- Keep services healthy in production: reliability, availability, performance, cost awareness
- Standardize **runbooks**, on-call processes, incident response, and rollback procedures
- Capacity & resilience: autoscaling policies, backups, safe config management, secrets handling
- Continuous operational work (patching, tuning) is part of the lifecycle
- Popular tools:** Kubernetes (HPA/Deployments); KMS (secrets); PagerDuty;

DevOps Stages (Monitor/Observability)

- Monitor what matters: **metrics, logs, traces** (and user experience signals)
- Define reliability targets using **SLIs/SLOs**; alert on symptoms, not noise
- Dashboards for visibility + alert routing to the right team with actionable context
- Monitoring output is a **feedback channel** to drive fixes, performance work, and prioritization
- **Popular tools:** Prometheus + Grafana; Nagios;

Who's in?

Just about everybody:

Amazon, Microsoft,
Google, Facebook,
Twitter, yahoo, Adobe,....



DevOps Global Market Trends

The DevOps market is expected to grow from an estimated \$10.4 billion in 2023 to \$25.5 billion in 2028.

Source:

<https://spacelift.io/blog/devops-statistics>

“DevOps is development and operations **collaboration**”

“DevOps is using **automation**”

“DevOps is **small** deployments”

It's DevOps!

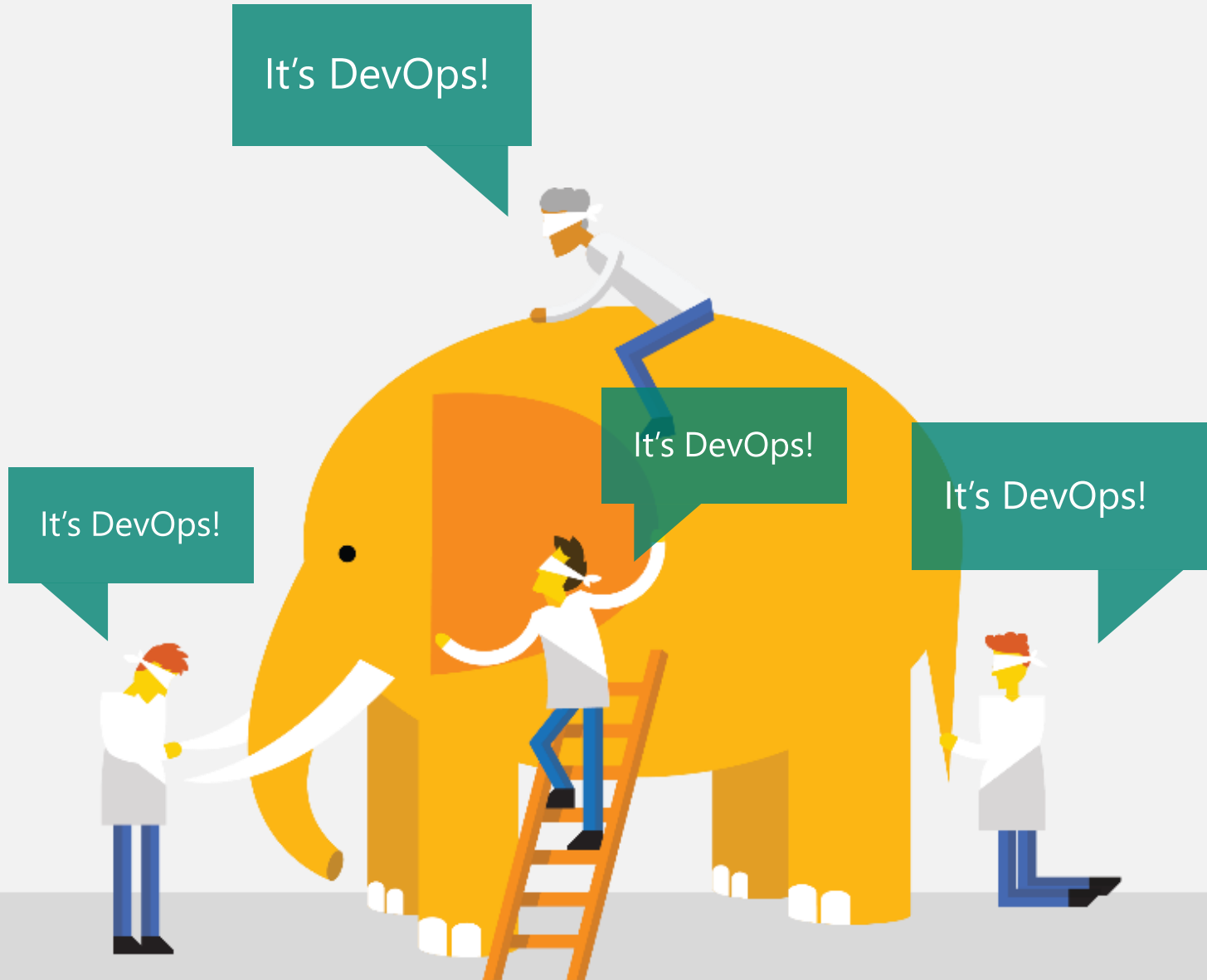
It's DevOps!

It's DevOps!

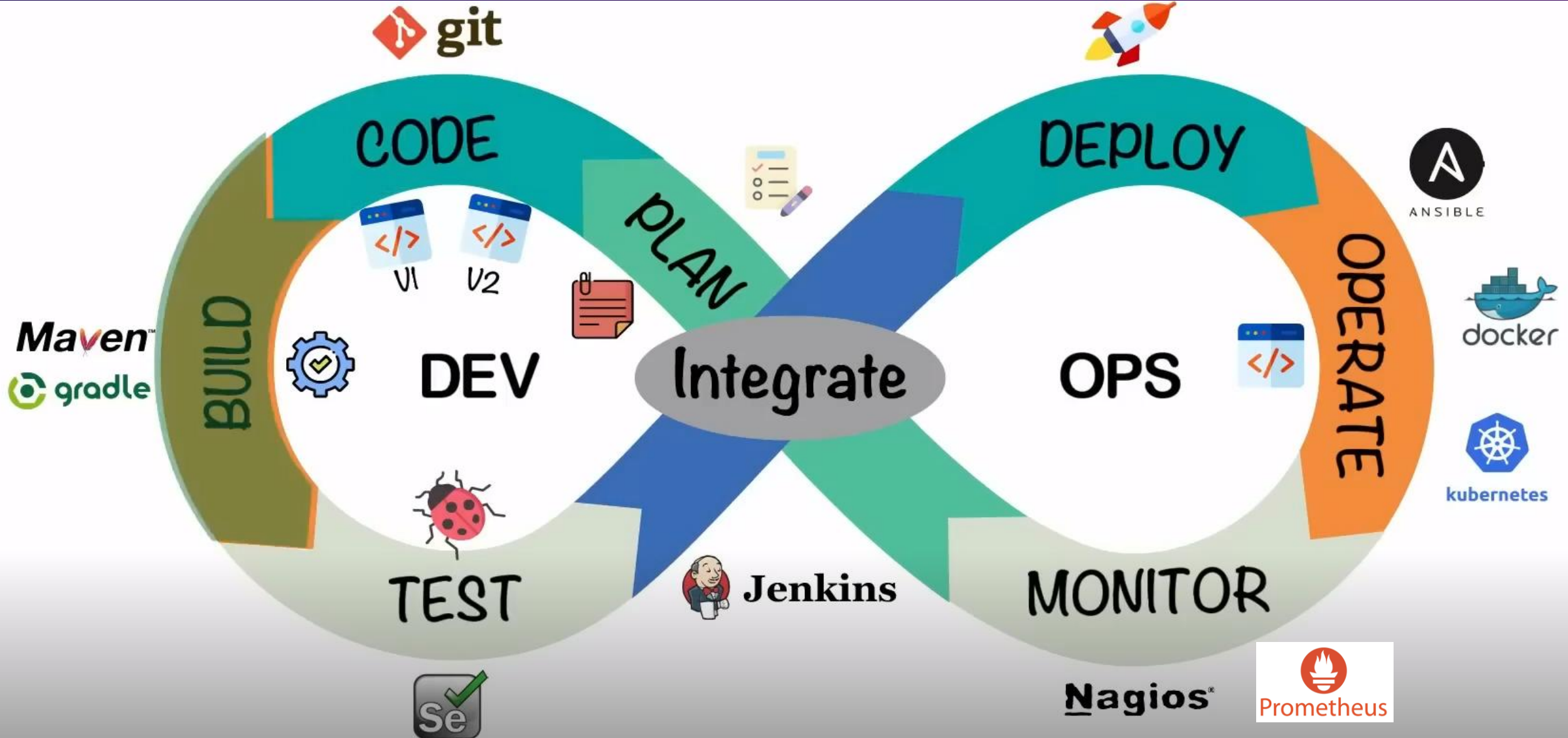
“DevOps is treating your **infrastructure as code**”

“DevOps is feature **switches**”

“**Agile** for Ops?”



Phases in DevOps Implementation



Yes, you want speed . . .





What you really
want is quality
at speed!

DevOps culture: CALMS

DevOps succeeds when culture and process change together (tools are secondary).



Culture

Trust, collaboration,
shared outcomes



Automation

Pipelines, IaC,
repeatability



Lean

Small batches,
remove waste



Measurement

Track delivery +
reliability



Sharing

Knowledge, visibility,
learning



If you only automate without culture change, you usually just ship problems faster.

What are the Features of DevOps Implementation?

- *"DevOps is not a goal but a never-ending process of continual improvement."*
- The DevOps offer continuous integration and continuous delivery (CI/CD). It makes the product delivery cycle quicker, and enterprises become able to launch the software timely without compromising its quality.

Following are the factors that will improve as a result of DevOps implementation:



- **Predictability:** DevOps decreases the failure rate of new product releases.
- **Maintainability:** The process improves the overall recovery rate at the time of the release event.
- **Improved Quality:** DevOps improves the quality of product development by incorporating infrastructure issues.
- **Lower Risk:** Security aspects are incorporated in SDLC, and the number of defects gets decreased across the product
- **Cost Efficient:** Cost efficiency is improved due to DevOps which is always an aspiration of every business organization.
- **Stability:** DevOps implementation offers a stable and secure operational state.
- **Streamlined Delivery Process:** As DevOps provides streamlined software delivery, the marketing effort is reduced up to 50%. It happens due to mobile applications and digital platforms.

To understand DevOps better you need to know it consists of three fundamental ideas.



**Practices and
Process**



**People and
Culture**

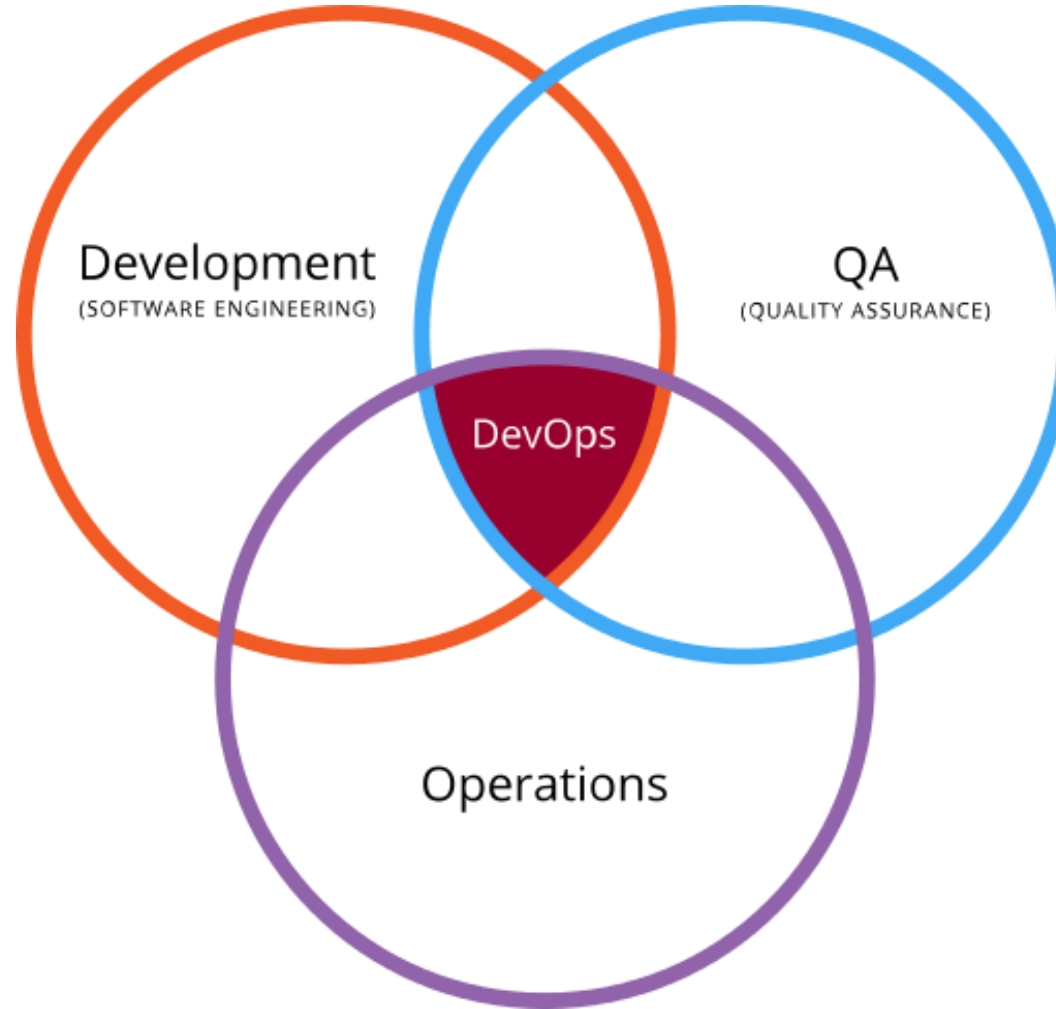


**Tools and
Techniques**

Source:

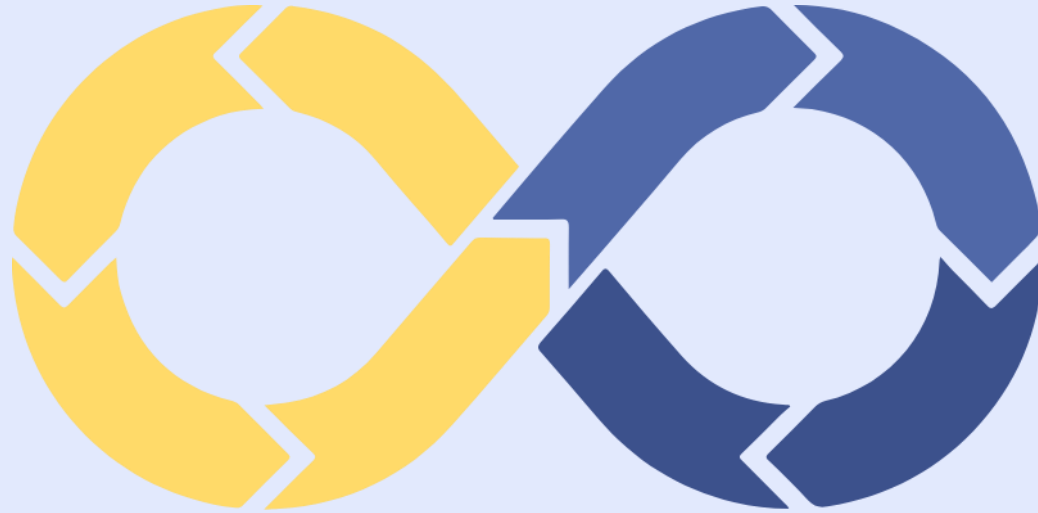
<https://www.weblinedia.com/blog/all-about-devops-automation/>

DevOps



DevOps is the **intersection** of **development** (software engineering), **operations** and **quality assurance** (QA)

What is Automation in DevOps?



Automation in DevOps makes the entire process **smooth**, and all conventional manual software development barriers become **scalable** and **manageable**. Furthermore, it helps to efficiently manage and run an application in the development environment with **frequent updates**. It also needs strict testing standards.

Source:

<https://www.weblineindia.com/blog/all-about-devops-automation/>

To sum it up



Benefits of DevOps Automation



**Removes
Manual
Error**



**Shorter
Development
Cycle**



**Latency
Removed**



**Increases
No.Of
Deliveries**



**Improved
Defect
Detection**



**Reduced
Deployment Failures
and Rollbacks**



**Reduced Time
to Recover Upon
Failure**

Source:

<https://www.weblinedia.com/blog/all-about-devops-automation/>